Software Design Document (SDD)

Lab Assignment: LM35 Temperature Sensor Driver

LM35 Driver — STM32F411RE

Module	LM35 Driver
Version	1.0.0
Team	Vania Yareni Leal Espinosa, Melanie Picen Hernandez, Steven Alexander McClellan Castro, Leonardo Ivan Garcia Espinosa
Date	30/05/2026
Institution	CETYS Universidad

1. Introduction

1.1 Purpose

This document describes the design and architecture of the LM35 temperature sensor driver module for the STM32F411RE microcontroller. It provides details on data structures, hardware signal mappings, control flow, and the public API to guide implementation and testing of higher-level software modules that rely on temperature sensing functionality.

1.2 Scope

The LM35 driver is a low-level software component for the STM32F411RE embedded system. It abstracts the analog temperature measurement process by combining GPIO configuration, ADC acquisition, multi-sample averaging, and a software calibration mechanism. The driver exposes three public functions — lm35_init, lm35_calibrate, and lm35_read — that together allow any higher-level module to obtain a compensated temperature reading in tenths of degrees Celsius without direct interaction with the ADC or GPIO peripherals.

1.3 Definitions and Acronyms

Term	Definition
LM35	Precision centigrade temperature sensor by Texas Instruments. Outputs 10 mV per °C.
ADC	Analog-to-Digital Converter — converts the analog voltage from the LM35 output into a digital value.
GPIO	General Purpose Input/Output — configurable digital/analog pins on the STM32F411RE.

STM32F411RE	ARM Cortex-M4 based microcontroller by STMicroelectronics used as the target platform.
Vref	ADC reference voltage (3.3 V). Determines the full-scale voltage range of the ADC.
12-bit ADC	ADC resolution used: converts voltage to an integer between 0 and 4095.
tempX10	Temperature encoded as an integer equal to (temperature in °C) × 10, to avoid floating-point types.
Calibration	Software process of computing an offset between the sensor reading and a known reference temperature.
RCC	Reset and Clock Control — peripheral responsible for enabling clocks to GPIO and ADC peripherals.
SRS	Software Requirements Specification — document defining the functional requirements for this module.
SDD	Software Design Document — this document.
API	Application Programming Interface — set of public functions exposed by the driver.
FR	Functional Requirement — a requirement that specifies a system behavior.

2. Requirements

All functional requirements specific to this module are defined in the Software Requirements Specification (SRS) for the LM35 driver module. The table below summarizes the traceability between each functional requirement and the corresponding public API function implemented in this driver.

Req. ID	Function	Requirement Statement
FR-1	lm35_init()	The system shall initialize the GPIO pin and ADC channel connected to the LM35 sensor.
FR-2	lm35_calibrate()	The system shall acquire multiple samples at a known ambient temperature and compute a calibration offset.
FR-3	lm35_read()	The system shall acquire multiple samples, convert to temperature, apply the calibration offset, and return the compensated result.

The SRS also defines the following non-functional requirements applicable to this driver: NFR-1 (no floating-point arithmetic in the public interface — integer tenths of degrees), NFR-2 (null-pointer validation on all public functions), and NFR-3 (multi-sample averaging to reduce ADC noise).

3. System Architecture

3.1 High-Level Design

The LM35 driver follows a layered software architecture that separates application logic from low-level hardware access. The three layers are described below:

Layer	Description
Application Layer	Higher-level modules (e.g., main, display, decision logic) that call lm35_init, lm35_calibrate, and lm35_read without any direct register access.
LM35 Driver Layer	Implements the public API declared in lm35_driver.h. Validates all input parameters, delegates GPIO and ADC initialization to their respective drivers, performs multi-sample averaging, and applies the calibration offset.
Hardware Layer	Actual STM32F411RE GPIO and ADC1 peripheral registers accessed through the GPIO driver (GPIO_stm32f4.h) and ADC driver (ADC_Driver.h) abstractions.

3.2 Module Interfaces

The LM35 driver exposes its functionality exclusively through the public header file lm35_driver.h. The module has the following interface characteristics:

- Public API: lm35_init, lm35_calibrate, and lm35_read declared in lm35_driver.h.
- Internal implementation: The static helper functions lm35_averageSamples and lm35_rawToCelsiusX10 are defined in lm35_driver.c and are not visible to external modules.
- Dependencies: The driver depends on ADC_Driver.h (for ADC_RegDef_t and adc_* functions) and GPIO_stm32f4.h (for gpio_initPort and gpio_setPinMode). No HAL or external libraries are used.
- Hardware dependencies: STM32F411RE GPIO port A pin 1 (PA1) configured as analog input, and ADC1 channel 1. The LM35 output pin must be connected to PA1.

4. Detailed Design

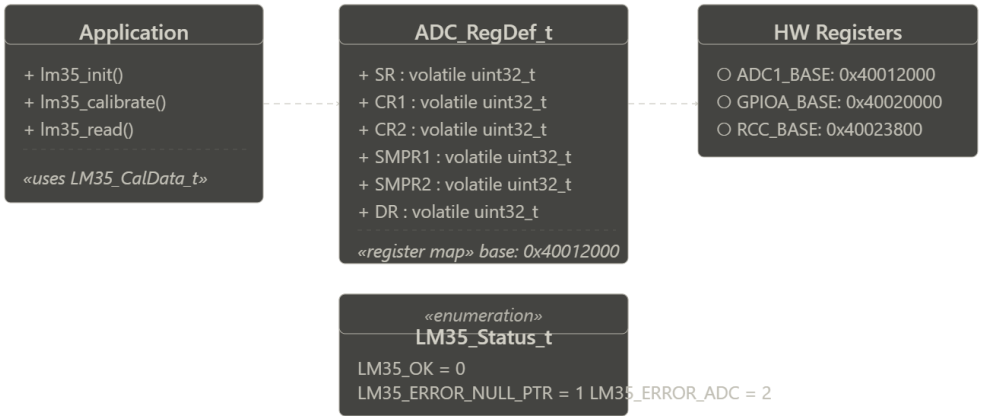

Figure 1. Class Diagram — LM35 Driver

4.1 Module Behavior

The following subsections describe the behavioral design of each public function, including its internal control flow and interactions with the underlying GPIO and ADC drivers.

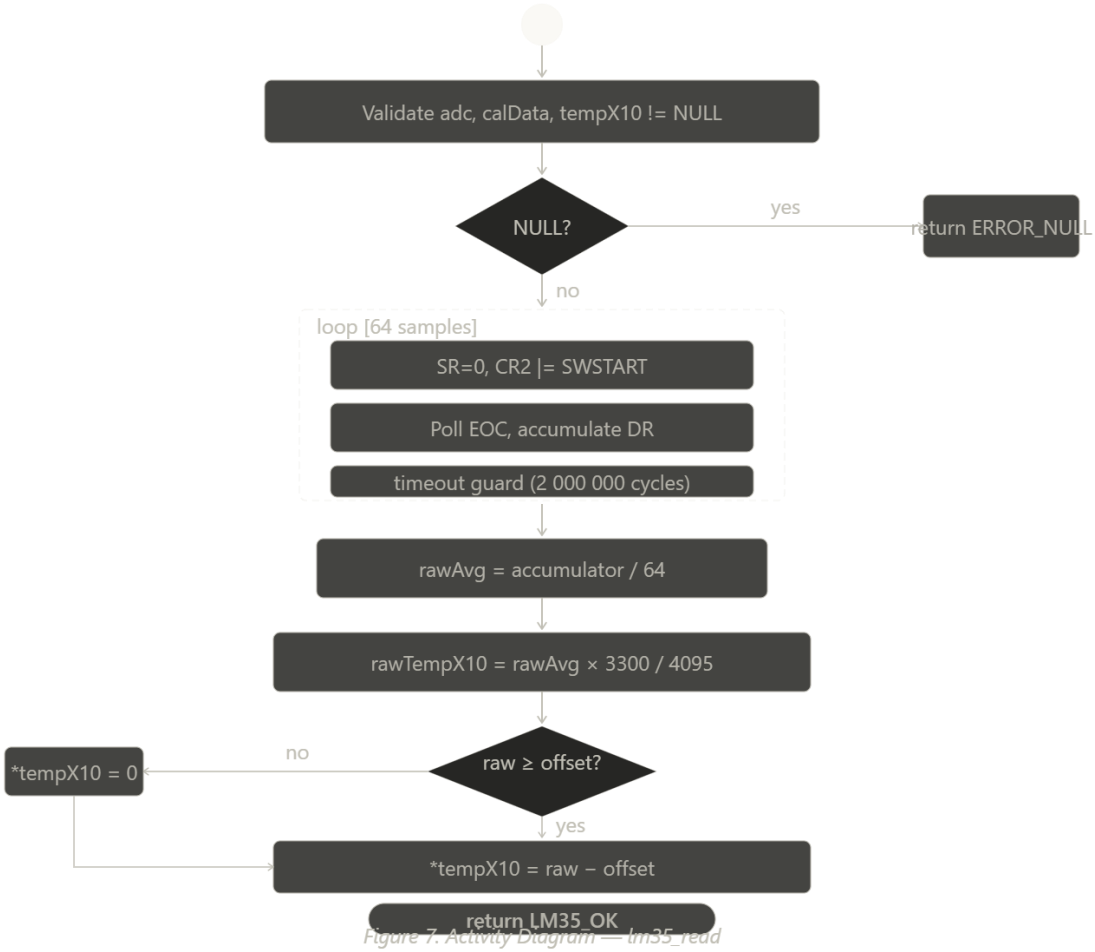

Figure 7. Activity Diagram — lm35_read

Figure 2. Sequence Diagram — lm35_init

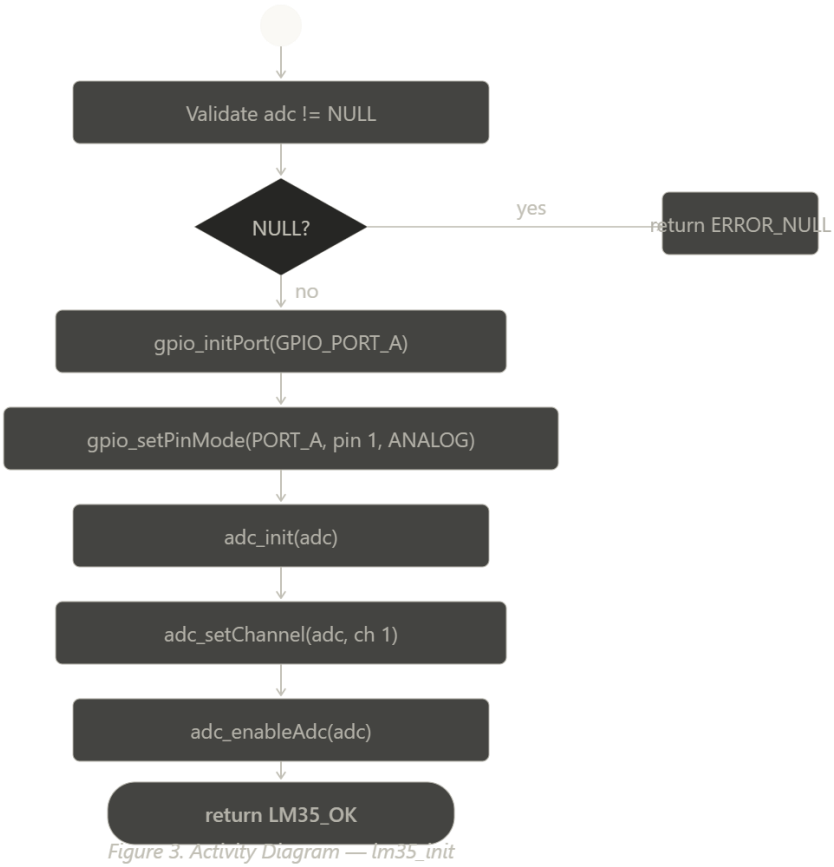

Figure 3. Activity Diagram — lm35_init

Figure 3. Activity Diagram — lm35_init

lm35_init — GPIO and ADC Initialization (FR-1)

The function first validates the adc pointer. It then calls gpio_initPort to enable the GPIOA clock, followed by gpio_setPinMode to configure PA1 in analog mode (GPIO_MODE_ANALOG). Next it calls adc_init to configure the ADC peripheral, adc_setChannel to select channel 1, and adc_enableAdc to power on the ADC with its built-in stabilization delay. Any failure in the GPIO or ADC sub-calls causes the function to return the corresponding error code immediately.

lm35_calibrate — Offset Computation (FR-2)

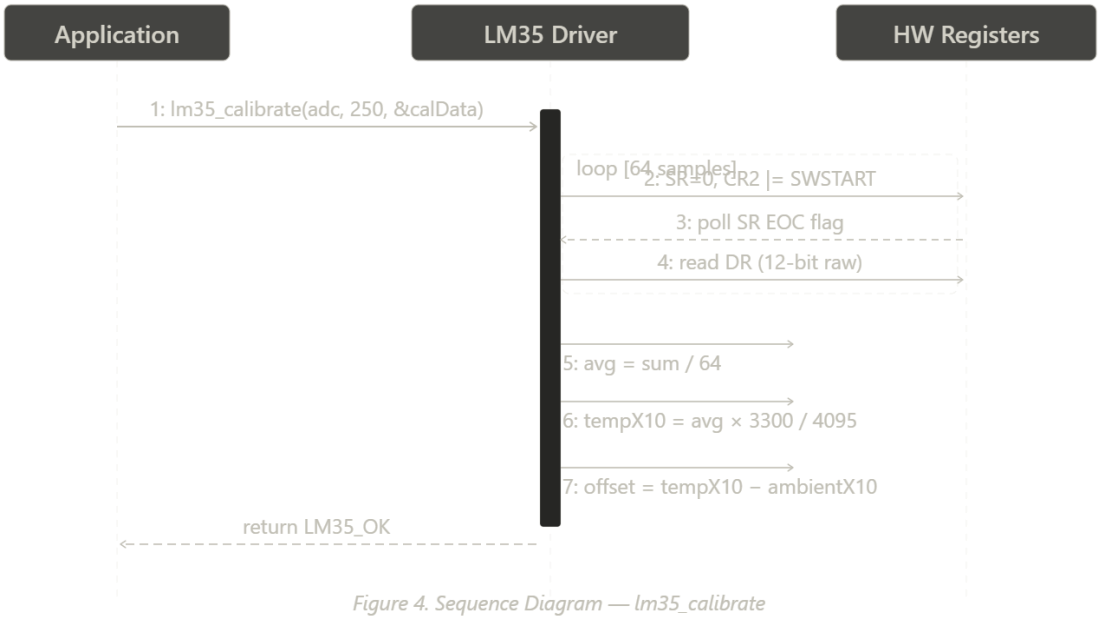

Figura 4 - Sequence lm35_calibrate

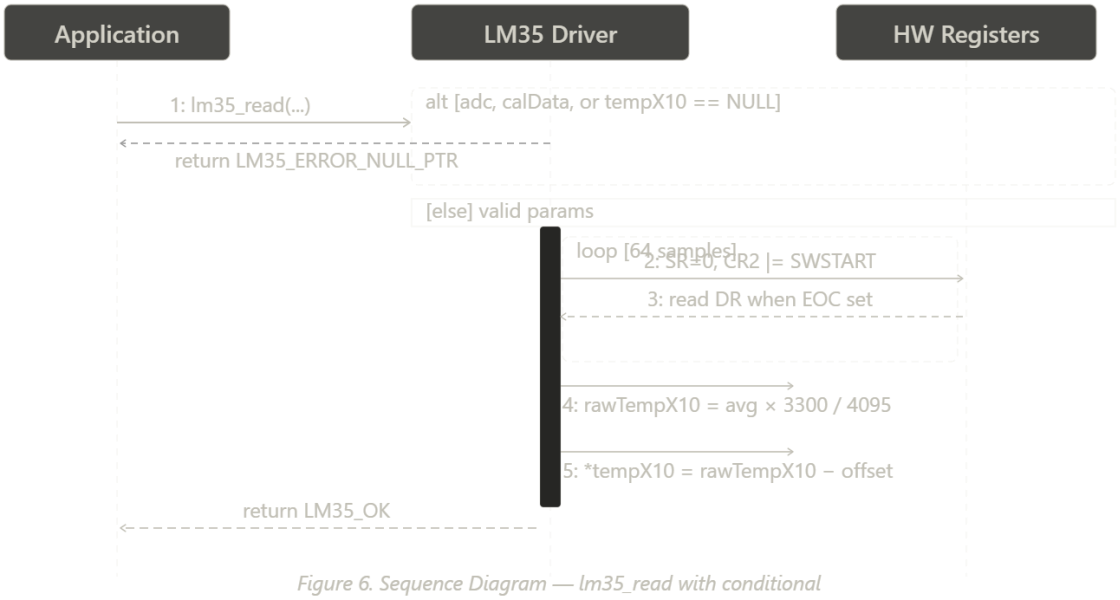

Figura 5 - Activity lm35_calibrate

The function validates both the adc and calData pointers. It then calls the internal helper lm35_averageSamples to collect LM35_NUM_SAMPLES (64) single-conversion ADC readings and compute their integer average. The average is converted to a temperature value in tenths of degrees Celsius via lm35_rawToCelsiusX10. The calibration offset is then computed as the difference between the measured value and the caller-supplied ambient reference (ambientTempX10). To prevent unsigned

underflow, the offset is only stored if the measured value is greater than or equal to the reference; otherwise it is set to zero.

lm35_read — Compensated Temperature Read (FR-3)

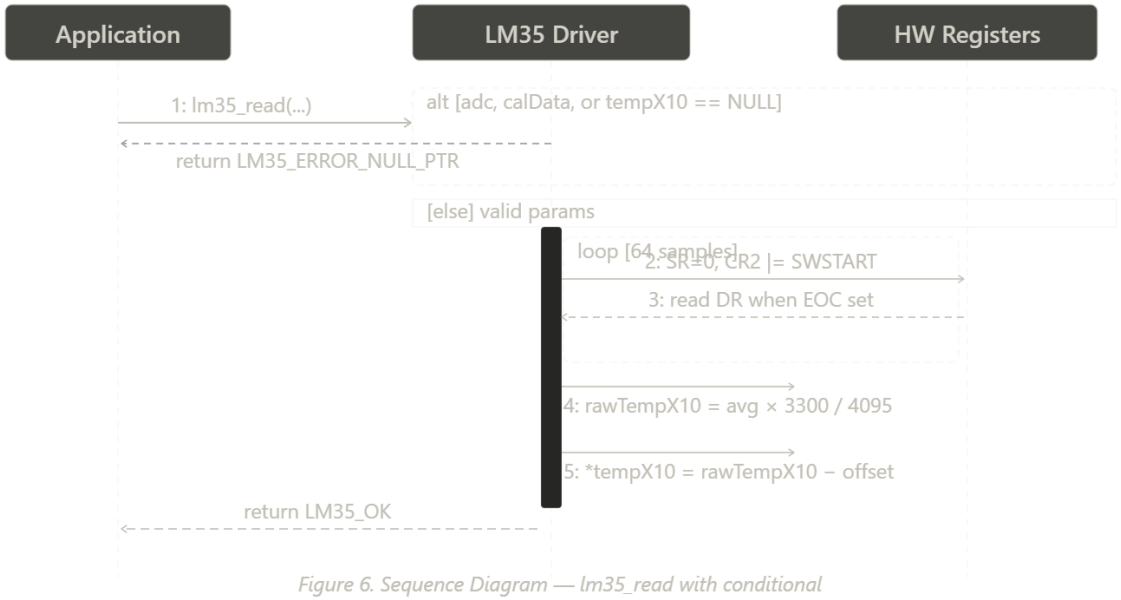

Figure 6. Sequence Diagram — lm35_read with conditional

Figura 6 - Sequence lm35_read

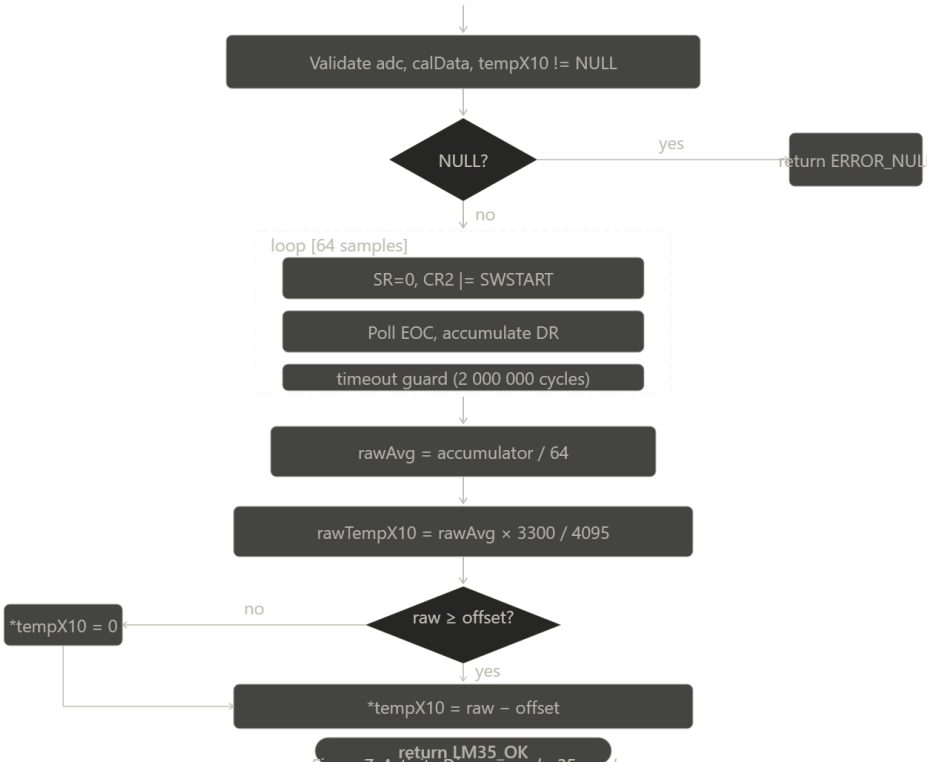

Figura 7 - Activity lm35_read

The function validates all three pointers (adc, calData, tempX10). It calls lm35_averageSamples to acquire 64 samples and converts the average to tenths of degrees using lm35_rawToCelsiusX10. The calibration offset stored in calData->ambientOffsetX10 is then subtracted from the raw result. As in lm35_calibrate, unsigned underflow is prevented by returning zero if the raw value is smaller than the offset.

Internal Helper: lm35_averageSamples

This static function triggers nSamples consecutive single ADC conversions by writing directly to the ADC SR and CR2 registers. Each conversion is started by setting the SWSTART bit (CR2 bit 30). The function polls the EOC flag (SR bit 1) with a timeout counter of 2 000 000 cycles; if the conversion completes within the timeout the 12-bit result is read from DR and added to the accumulator. The integer average is returned. This function is not visible outside lm35_driver.c.

Internal Helper: lm35_rawToCelsiusX10

This static function converts a 12-bit ADC average to a temperature in tenths of degrees Celsius using integer arithmetic only. The formula applied is: tempX10 = (rawAvg × 3300) / 4095, which is derived from Vout(mV) = rawAvg × (3300 / 4095) and T(°C) = Vout(mV) / 10, combined as T × 10 = rawAvg × 3300 / 4095.

4.2 Driver State Machine

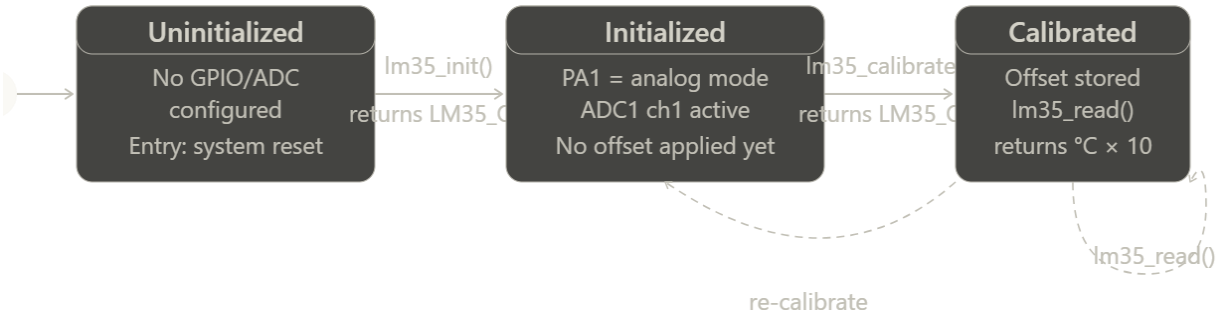

Figura 8 - State Diagram

The LM35 driver follows a three-state lifecycle from system reset through to active measurement:

State	Description	Transition
Uninitialized	Driver has not been configured. No GPIO or ADC registers have been touched.	lm35_init() returns LM35_OK → moves to Calibrated (pending).
Initialized	GPIO PA1 is in analog mode and ADC1 channel 1 is active. The sensor can be read but no offset has been applied.	lm35_calibrate() returns LM35_OK → moves to Calibrated.

Calibrated	Calibration offset is stored in LM35_CalData_t. lm35_read() returns fully compensated values.	lm35_read() called repeatedly. Re-calibrate at any time by calling lm35_calibrate() again.

5. Application Programming Interface

5.1 Data Types and Macros

Preprocessor Macros

Macro	Value	Definition
LM35_GPIO_PORT	GPIO_PORT_A	GPIO port where the LM35 output pin is connected.
LM35_GPIO_PIN	1U	GPIO pin number on port A connected to the LM35 output (PA1).
LM35_ADC_CHANNEL	1U	ADC1 channel that maps to PA1 (ADC1_IN1).
LM35_NUM_SAMPLES	64U	Number of ADC samples averaged per calibration or read call.
LM35_ADC_FULL_SCALE	4095U	Maximum ADC output for 12-bit resolution (2^12 − 1).
LM35_VREF_MV	3300U	ADC reference voltage in millivolts (3.3 V).

Enumerations

LM35_Status_t: Return codes for all public LM35 driver functions.

Enumerator	Value	Definition
LM35_OK	0	Operation completed successfully.
LM35_ERROR_NULL_PTR	1	A required pointer argument was NULL.
LM35_ERROR_ADC	2	An underlying ADC driver operation returned a non-OK status.
LM35_ERROR_GPIO	3	An underlying GPIO driver operation returned a non-OK status.

Structures

LM35_CalData_t: Calibration data produced by lm35_calibrate and consumed by lm35_read.

Member	Type	Definition
ambientOffsetX10	uint32_t	Calibration offset in tenths of °C. Equals (sensor reading at calibration time) − (known ambient

		temperature), both expressed as tempX10 values. Subtracted from every lm35_read result.

5.2 Error Codes

All public LM35 API functions return LM35_Status_t. The table below lists all defined error codes:

Code	Value	Definition
LM35_OK	0	Operation completed successfully.
LM35_ERROR_NULL_PTR	1	A required pointer is NULL.
LM35_ERROR_ADC	2	ADC initialization, channel selection, or enable step failed.
LM35_ERROR_GPIO	3	GPIO port init or pin mode configuration failed.

5.3 Public Function Definitions

lm35_init

Function Name	lm35_init
Syntax	LM35_Status_t lm35_init(ADC_RegDef_t *adc)
Parameters (in)	adc — ADC_RegDef_t* — pointer to the ADC1 peripheral register structure.
Parameters (inout)	None
Parameters (out)	None
Return value	LM35_Status_t — LM35_OK on success. LM35_ERROR_NULL_PTR if adc is NULL. LM35_ERROR_GPIO if GPIO init or pin mode fails. LM35_ERROR_ADC if ADC init, channel set, or enable fails.
Description	Enables GPIOA clock via gpio_initPort, configures PA1 in analog mode via gpio_setPinMode, initializes ADC1 via adc_init, selects channel 1 via adc_setChannel, and powers on the ADC via adc_enableAdc. All sub-steps are validated; the first failure causes immediate return with the corresponding error code.
Constraints	Must be called before lm35_calibrate or lm35_read. The LM35 output pin must be physically connected to PA1.
Available via	lm35_driver.h

lm35_calibrate

Function Name	lm35_calibrate

Syntax	LM35_Status_t lm35_init(Adc_Instance_t instance) , uint32_t ambientTempX10, LM35_CalData_t *calData)
Parameters (in)	adc — ADC_RegDef_t* — pointer to the ADC1 peripheral register structure.
Parameters (in)	ambientTempX10 — uint32_t — known ambient temperature in tenths of °C (e.g., 250 for 25.0 °C).
Parameters (inout)	None
Parameters (out)	calData — LM35_CalData_t* — pointer to a caller-allocated struct that receives the computed offset.
Return value	LM35_Status_t — LM35_OK on success. LM35_ERROR_NULL_PTR if adc or calData is NULL.
Description	Acquires LM35_NUM_SAMPLES (64) ADC readings via lm35_averageSamples and converts the average to tenths of °C via lm35_rawToCelsiusX10. Computes and stores ambientOffsetX10 = measuredX10 - ambientTempX10, or 0 if the measured value is smaller, to prevent unsigned underflow.
Constraints	lm35_init must be called before this function. The sensor should be at a known stable ambient temperature during calibration. Should be called once at system startup.
Available via	lm35_driver.h

lm35_read

Function Name	lm35_read
Syntax	LM35_Status_t lm35_read(ADC_RegDef_t *adc, const LM35_CalData_t *calData, uint32_t *tempX10)
Parameters (in)	adc — ADC_RegDef_t* — pointer to the ADC1 peripheral register structure.
Parameters (in)	calData — const LM35_CalData_t* — pointer to calibration data from lm35_calibrate.
Parameters (inout)	None
Parameters (out)	tempX10 — uint32_t* — pointer where the compensated temperature in tenths of °C is stored.
Return value	LM35_Status_t — LM35_OK on success. LM35_ERROR_NULL_PTR if any pointer is NULL.
Description	Acquires 64 ADC samples, averages them, converts to tenths of °C, subtracts calData->ambientOffsetX10, and stores the result at *tempX10. The result is clamped to 0 if the raw reading is smaller than the offset to prevent unsigned underflow. Can be called repeatedly in the main loop.
Constraints	lm35_init and lm35_calibrate must be called before this function. calData must point to a valid, previously populated LM35_CalData_t.

Available via	lm35_driver.h

5.4 Internal Function Definitions

The following static helper functions are defined in lm35_driver.c and are not accessible from outside the module.

Function	Signature	Description
lm35_averageSamples	static uint32_t lm35_averageSamples(ADC_RegDef_t *adc, uint32_t nSamples)	Triggers nSamples single ADC conversions via direct register writes to SR and CR2 SWSTART. Polls EOC with a timeout. Returns the integer average of all successful readings.
lm35_rawToCelsiusX10	static uint32_t lm35_rawToCelsiusX10(uint32_t rawAvg)	Converts a 12-bit ADC average to tenths of °C using integer arithmetic: (rawAvg × 3300) / 4095. No floating-point operations used.

6. Future Enhancements

The following enhancements may be considered for future revisions of this LM35 driver:

- Signed temperature support: Extend the data representation to int32_t to support sub-zero temperatures, which the LM35DZ variant can measure down to −55 °C.
- Configurable channel and pin: Replace the LM35_ADC_CHANNEL and LM35_GPIO_PIN constants with parameters in lm35_init to allow multiple LM35 sensors on different channels.
- Configurable sample count: Allow the caller to specify the number of averaging samples to trade off between speed and noise rejection.
- Interrupt-driven acquisition: Replace the polling-based lm35_averageSamples with an interrupt or DMA-driven approach to avoid blocking the CPU during conversion.
- Temperature alarm callback: Add an optional threshold mechanism that invokes a user-supplied callback when the temperature exceeds a configurable limit.
- Persistent calibration storage: Provide hooks to save and restore the LM35_CalData_t offset to/from non-volatile memory (e.g., internal Flash or EEPROM) so calibration survives power cycles.